

Session 2: Testing Planning

(Test Plan)

Chapter 1: What is a Test Plan and why is it vital?

1. Definition of Test Plan (Test Plan)
2. Why do we need a Test Plan? (Justification of the investment)
3. The difference between Testing Strategy and Testing Plan

💡 Practical Examples for Discussion

Did you know that...?

Applied Story: "Planning an Everest Expedition"

🧐 Reflection activity: "What do we put in our backpack?"

Chapter 2: Standard Structure of a Test Plan (Sectional Analysis)

1. Objective and Scope
2. Roles & Responsibilities
3. Schedule & Milestones
4. Entry & Exit Criteria
5. Suspension and Resumption Criteria

💡 Practical Examples for Discussion

Did you know that...?

Applied Story: "Organizing a Wedding Event"

🧐 Thought Exercise: "OrangeHRM in Action"

Chapter 3: Business Requirements Analysis and Derivation of Test Conditions

1. What are Business Requirements?
2. Why do we analyze the requirements exhaustively?
3. Derivation of Test Conditions

💡 Practical Examples for Discussion (OrangeHRM)

Did you know that...?

Applied Story: "The Chef's Recipe and the Pretentious Customer"

🧐 Thinking Exercise: "The Requirements Detective"

Chapter 4: Test Case Design (Anatomy and Writing of Test Cases)

1. What is a Test Case?

2. The Comprehensive Structure of a Test Case (Anatomy)

- A. Test Case ID (Unique Identifier)
- B. Title / Summary
- C. Precondiții (Preconditions)
- D. Test Steps
- E. Test Data
- F. Expected Result - The most important field!
- G. Real Result (Actual Result)
- H. Post-conditions

3. Properties of a Quality Test Case (QualiAdept Standards)

💡 Practical Examples for Discussion (OrangeHRM)

Did you know that...?

Applied Story: "The IKEA Assembly Manual"

🧐 Thinking Exercise: "Let's write correctly!"

Chapter 5: Types of Tests and the Testing Pyramid

1. Testing Levels

- A. Unit Testing
- B. Integration Testing
- C. System Testing
- D. Acceptance Testing (UAT)

2. The Testing Pyramid

3. Retesting vs. Regression Testing (The Big Distinction)

💡 Practical Examples for Discussion (OrangeHRM)

Did you know that...?

Applied Story: "Building a Smartphone"

🧐 Thinking Exercise: "The Regression Detective"

Bonus Chapter: Exploratory Testing vs. Scripted Testing (Intuition vs. Structure)

1. Scripted Testing

2. Exploratory Testing

🔗 Comparison: When do we use what?

💡 Practical Examples for Discussion (OrangeHRM)

Did you know that...?

Applied Story: "The Tourist vs. the Explorer"

🧐 Thinking Exercise: "Beyond the Steps"

Chapter 6: Recap, Conclusions and the OrangeHRM Project

1. Comprehensive Review: What did we learn in Session 2?
 2. QualiAdept Conclusions: "The Laws of the QA Strategist"
 3. Homework: "OrangeHRM Project - Design Phase" 🚀
 - Task 1: Test Plan Fragment (PIM Module - Personal Information Management)
 - Task 2: Requirements Analysis (Case "Add Employee")
 - Task 3: Test Case Design (Execution)
 - Task 4: Regression Strategy
- 💡 Final tip from the QualiAdept team

Chapter 1: What is a Test Plan and why is it vital?

🧠 In this chapter, we explore the central document that governs all QA activity in a project. If in Session 1 we learned that we are detectives, in Session 2 we learn to be "chiefs of operations." A Test Plan is not just a bureaucratic formality, it is the map that prevents us from getting lost in the complexity of the software and exceeding budgets or deadlines.

1. Definition of Test Plan (Test Plan)

Definition: The Test Plan is a detailed document that describes the strategy, objectives, schedule, resources (human and technical) and scope of testing activities.

- **Role of the document:** It serves as a "contract" between the testing team and the rest of the organization (Developers, Managers, Customers).
- **Who creates it?** Usually the Test Lead or a Senior QA, but the entire team contributes to the technical details.
- **When is it created?** In the project planning phase, immediately after the business requirements have been clarified.

2. Why do we need a Test Plan? (Justification of the investment)

Without a plan, testing becomes a subjective and inefficient process. Here are the exhaustive reasons why no serious company starts testing without a Plan:

- **Clarification of Scope:** Define exactly **what are we testing** and, equally important, **what**

we do **NOT** test (Ex: We are testing the web version, but not the mobile version).

- **Resource Identification:** We know how many people we need, what phones/laptops we need to buy, and what database access we need.
- **Risk assessment:** We anticipate problems (e.g., "If the test environment goes down, how do we recover lost time?").
- **Time Control:** Set clear milestones. We know exactly what day we start and what day we have to give the final verdict.
- **Traceability and Transparency:** Anyone on the project can open the Plan and see exactly the quality status.

3. The difference between Testing Strategy and Testing Plan

This is a classic interview trap question.

- **Test Strategy:** It's a high-level document, usually company-wide, that says "how we test in general" (e.g. "all our projects use Jira and require 80% coverage"). It's static.
- **Test Plan:** It is specific to a project. It says "how we are testing the OrangeHRM application in this month's sprint." It is dynamic and updates as the project evolves.

Practical Examples for Discussion

Scenario A: The Project without the "Map" (No Plan)

- **Context:** A team of 5 QA's get a new app. The manager says, "Just find bugs, don't waste time on documentation."
- **Consequence:** After 2 weeks, 3 testers checked the same Login page, no one checked payment processing, and the test environment was deleted by a programmer because he didn't know QA needed it.
- **QA Analysis:** The test plan would have assigned different tasks to each tester and "reserved" the test environment, avoiding wasting time.

Scenario B: "What we DON'T test" is a lifesaver

- **Context:** The Test Plan clearly states: "Security testing (Penetration Testing) is not part of the scope of this project."
- **Consequence:** At the end of the project, a client asks why no hacking tests were done.
- **QA Analysis:** The Test Plan protects the QA team. If signed by the customer, they confirm that they have agreed to the limitations of the process.

Did you know that...?

💡 A Test Plan is considered a **Living Document** ? If during the project a new functionality is added that was not in the original plan, the document must be updated. If we ignore updating the plan, it becomes a "useless piece of paper" that no longer reflects the reality on the ground.

Applied Story: "Planning an Everest Expedition"

Imagine you want to climb Everest.

- **Chaotic approach:** You grab your backpack and head out. You might forget your oxygen, you might not have enough people to carry the tents, or you might get caught in a storm without a planned shelter. The chances of failure are 99%.
- **Test Plan Approach (Planned Expedition):**
 - **Scope:** We reach the top and return alive.
 - **Resources:** 5 Sherpas, 20 oxygen tanks, food for 30 days.
 - **Risks:** Avalanches, frostbite. Back-up plan: If the weather is bad, we wait in Camp 2.
 - **Entry criteria:** We don't start the climb until we receive a favorable weather forecast.

QualiAdept's conclusion: The Test Plan is that expedition planning. It doesn't climb the mountain for you (it doesn't run the tests), but it makes sure that when you're on the mountain, you have everything you need so you don't fail.

Reflection activity: "What do we put in our backpack?"

Think about the app **OrangeHRM**. If you should plan to test the Leave Management module:

- Which would be the biggest **Risk**? What can you think of?
- This **Resources** technical requirements do you think you would need? (Ex: Database access, Manager account access and Employee account access).

Chapter 2: Standard Structure of a Test Plan (Sectional Analysis)

 A professional Test Plan follows a logical structure, often based on the international standard IEEE 829 (or modern adaptations of it). Each section plays a critical role in eliminating ambiguity. We will break this document down into its most important “bodies”, explaining in detail what each should contain.

1. Objective and Scope

This is the “boundary” of your project. Without a well-defined Scope, the QA team risks testing too much or too little.

- **In-Scope:** List of functionalities that **will be** tested.
 - *OrangeHRM example:* "User Management Module, Leave Management Module, Login and Password Reset."
- **Out-of-Scope:** List of functionalities that **they will not be** tested. It is vital to write this to manage customer expectations.
 - *OrangeHRM example:* "Performance testing on over 10,000 simultaneous users, Testing on Internet Explorer browsers (outdated), Recruitment Module (which has not yet been delivered)."

2. Roles & Responsibilities

Who does what? In a QualiAdept team, clarity of roles prevents "I thought my colleague was doing it" situations.

- **Test Manager/Lead:** Plan, monitor risks, sign the final document.
- **QA Engineer (Tester):** Write test cases, run tests, report bugs.
- **Developer:** Fixes reported bugs and delivers new versions (builds).

- **Product Owner (PO):** Clarify business requirements and decide priorities.

3. Schedule & Milestones

Testing cannot go on forever. We need to have clear timelines.

- **Milestone 1:** Test Plan Completion (Date: X).
- **Milestone 2:** Completion of writing Test Cases (Date: Y).
- **Milestone 3:** Execution completion (Date: Z).
- **Milestone 4:** Final test report (Date: W).

4. Entry & Exit Criteria

These are the “toll gates” of the testing process. They tell us when we are allowed to start and when we are allowed to stop.

- **Entry Criteria:** What needs to be ready to start testing?
 - *Example:* Approved test plan, Functional test environment, Code delivered by developers (without compilation errors).
- **Exit Criteria:** When do we consider we are done?
 - *Example:* All planned tests have been executed, 100% of Critical and Major bugs are fixed and closed, Requirements Coverage is 100%.

5. Suspension and Resumption Criteria

What do we do if the “film breaks”?

- **Suspension:** We stop testing if a major bottleneck occurs.
 - *Example:* The OrangeHRM application crashes immediately after logging in. We cannot test the remaining modules.
- **Resume:** When do we start again?
 - *Example:* After the developers deliver a documented fix for the issue that caused the suspension.

Practical Examples for Discussion

Scenario A: “The Mystery of the Unreported Bug”

- **Context:** A tester finds a performance bug (the application is running slowly). The manager scolds him for wasting 4 hours investigating it.
- **QA Analysis:** The tester checks the Test Plan at the section **Scope**. It says there:

"Performance Testing is Out-of-Scope."

- **Conclusion:** The tester made a mistake by wasting resources on something that wasn't planned. The plan helps us stay focused on priorities.

Scenario B: "Launch Pressure"

- **Context:** The client wants to launch the app on Friday. We have 10 open bugs, 2 of which are Critical.
- **QA Analysis:** We check **Exit Criteria** It clearly says there: "Cannot launch with open Critical bugs."
- **Conclusion:** The Test Plan is your legal argument to the client. You don't say "I don't want to launch", you say "I didn't meet the agreed-upon exit criteria".

Did you know that...?

💡 In Agile projects, the Test Plan is much shorter and often focused on a single Sprint (a 2-week period)? We no longer write 50-page novels, but synthetic documents called **Test Summary** or "Lean" level plans, to keep up with the speed of development.

Applied Story: "Organizing a Wedding Event"

Imagine you are the wedding planner (QA Lead). Your test plan is the "Event Manager".

- **Scope:** Religious ceremony, Meal, Music. (Out-of-Scope: Transportation of guests from the airport - someone else takes care of that).
- **Roles:** The Chefs (Devs) cook, the Waiters (Testers) taste and check if the food is hot, the Groom (Product Owner) decides the order of the songs.
- **Entry Criteria:** We don't start the meal until all the guests have arrived and the food has been delivered by the supplier.
- **Suspension Criteria:** If the power goes out (Blockage), the party is suspended until the maintenance team arrives.
- **Exit Criteria:** The wedding is considered over when the cake has been cut and the guests have received their leaving gifts.

Without this scheduler, the chefs could be cooking the second course before the appetizer, or the music could start while the groom is still at the church. In QA, the Test Plan ensures that the "party" (software release) is a success, not a mess.

Thought Exercise: "OrangeHRM in Action"

Imagine you need to define the Exit Criteria for testing the module **Recruitment** from OrangeHRM:

- What percentage of tests must be "Passed"?
- What do we do with typos? Do we let them pass or block the release?
- What is the minimum technical condition for a candidate to be able to say that the testing process is complete?

Chapter 3: Business Requirements Analysis and Derivation of Test Conditions

💰 An elite tester doesn't wait to receive the application to start working. He starts testing as soon as he receives the documentation. Requirements analysis is a form of **Static Testing** where we "hunt" for logic bugs before they are programmed. If the requirement is ambiguous, the code will be wrong, and your test will be confusing. In this chapter, we learn how to "dissect" a requirement to extract **Testing Conditions**.

1. What are Business Requirements?

Requirements are descriptions of what the system must do to satisfy a user need. They can come in various forms:

- **User Stories (in Agile):** "As a user, I want to be able to reset my password to regain access to my account."
- **SRS (Software Requirement Specification):** Detailed technical documents with diagrams and strict rules.
- **Mockups/Wireframes:** The visual design of the pages.

2. Why do we analyze the requirements exhaustively?

Our objective is to find **documentation defects**. A bad requirement is the "root" of 50% of production bugs. We look for:

- **Ambiguity:** Words like "fast", "easy", "sometimes", "approximately". (Ex: "The system should load quickly" – What does fast mean? 1 second or 10?).
- **Incompleteness:** What happens in case of error? (Ex: The requirement says how to log in, but doesn't say what happens if you enter the wrong password 5 times).
- **Inconsistency:** When two requirements clash. (Ex: Page A says the font is red, Page B says the font is blue).

3. Derivation of Test Conditions

Testing Condition is an element or event that can be verified. It is the answer to the question: "WHAT are we testing?".

- **Requirement:** "The user must be able to log in to OrangeHRM with a valid username and password."
- **Derived Test Conditions:**

- Verifying login with valid data.
- Checking behavior when entering an incorrect username.
- Checking behavior when entering a wrong password.
- Checking case sensitivity.
- Access verification with empty fields.

💡 Practical Examples for Discussion (OrangeHRM)

Scenario A: The "Lazy" Requirement

- **Requirement:** "The Leave module should allow employees to request days off."
- **QA Analysis:** This requirement is a disaster for a tester.
- **Questions to ask during the course:**
 - What types of leave are there? (Medical, Rest, Unpaid?)
 - Who approves the request?
 - Can fractions of a day (hours) or only whole days be requested?
 - What happens if the employee has no more days available?

Scenario B: Traceability

- **Concept:** Each Test Condition must be linked to a Requirement.
- **Discussion:** If you have a test that doesn't check any requirements, it means you're testing something useless. If you have a requirement that has no associated test, it means that functionality is a major risk (untested).

Did you know that...?

💡 There is a technique called "**Intentional Ambiguity**"? Sometimes clients leave requirements vague because they themselves don't know exactly what they want the final product to look like. Your role as a QA is to "force" clarity. A timely question asked by a tester can save thousands of euros in programming hours wasted on a bad idea.

Applied Story: "The Chef's Recipe and the Pretentious Customer"

Imagine you are a food critic (QA) and you receive an order from a client for a chef (Developer).

- **Customer Requirement:** "I want some nice, warm soup."
- **Analiza ta (QA):** If you give this "requirement" to the chef, he might make a fish soup at 40 degrees. But what if the customer hates fish and wanted the soup at 80 degrees?
- **Your action:** You go to the client and analyze:
 - What does "good" mean? (Salty, spicy, chicken?)
 - What does "hot" mean? (Exact temperature).
 - Are there any allergies? (Negative conditions).

Only after you have "dissected" the requirement and written: "Chicken soup, with noodles, served at 75 degrees, without parsley", can the chef cook (Code), and you can check (Test) whether the result is as expected. Without this analysis, the chef works in vain, and the customer remains dissatisfied.

Thinking Exercise: "The Requirements Detective"

Analyze the following User Story for **OrangeHRM** and identifies 3 ambiguities or missing information:

"As an administrator, I want to be able to delete users from the system with a simple click, to clean up the database."

1. **Ambiguity 1:** _____
2. **Ambiguity 2:** _____
3. **Ambiguity 3:** _____

(Trainer Tips: What happens to the deleted user's historical data? Is there a confirmation message "Are you sure you want to delete?" or is it deleted instantly? Can an admin delete themselves?)

Chapter 4: Test Case Design (Anatomy and Writing of Test Cases)

 If the Test Plan is the "War Strategy", **Test Case** is the specific "Order of Battle". A test case is a set of conditions or variables under which a tester will determine whether a software

system meets the requirements or functions correctly. In the QualiAdept model, a well-written Test Case must be **atomic**, **independent** and **clear**.

1. What is a Test Case?

A Test Case is a document that describes a specific application usage scenario. It transforms a "Test Condition" (e.g., Login Verification) into a sequence of reproducible steps.

Golden rule: A test case must be able to be executed by anyone on the team, without requiring additional explanations from the author.

2. The Comprehensive Structure of a Test Case (Anatomy)

Each field in a Test Management tool (like Zephyr in Jira) has a specific purpose. Let's take a closer look at them:

A. Test Case ID (Unique Identifier)

It is an alphanumeric code that helps with traceability.

- *Recommended format* :[Project]_[Module]_[Number]
- *Example*: OHRM_LOGIN_001 (OrangeHRM, Login module, test 1).

B. Title / Summary

A short and clear description of what we are checking. It should contain "What", "Where", and sometimes "When".

- *Wrong*: "Login test".
- *Correct (QualiAdept style)*: "Verifying successful login using valid Administrator credentials."

C. Precondiții (Preconditions)

The state in which the system should be **before** to start the testing steps.

- *Example*: "The OrangeHRM application is open on the Login page", "The user 'Admin' is already registered in the database".

D. Test Steps

Clear, actionable instructions. Use imperative verbs (Press, Enter, Navigate).

- Enter the text 'Admin' in the 'Username' field.
- Enter the text 'admin123' in the 'Password' field.
- Click the 'Login' button.

E. Test Data

The exact values used during the test.

- *Example:* Username: Admin, Password: admin123.

F. Expected Result - The most important field!

The behavior that the system **should** to have it according to business requirements.

- *Example:* "The user is redirected to the Dashboard page. The 'Welcome Admin' welcome message appears."

G. Real Result (Actual Result)

Field to be completed only at the moment **IMPLEMENTATION**. If the Result is Real $\neq$ Expected Result, we have a Bug!

H. Post-conditions

System status after test completion (after cleanup).

- *Example:* "The user logs out to leave a clean environment for the next test."

3. Properties of a Quality Test Case (QualiAdept Standards)

To write professional tests, we must follow these principles:

- **Atomicity:** A test should only check one thing. Don't combine "Login" and "Change Password" in the same Test Case. If it fails, you won't know which part is at fault.
- **Independence:** The test should not depend on the result of the previous test.
- **Reproducibility:** Anyone running it, on any valid medium, should get the same result.
- **Traceability:** Each Test Case must be linked to a requirement (User Story) in Jira.

Practical Examples for Discussion (OrangeHRM)

Scenario A: The "Lazy" Test

- **Steps:** 1. Log in. 2. Check the Dashboard.
- **QA Analysis:** This test is too vague. What username do we use? What does "verify" mean? What should we see on the Dashboard? Such a test will be interpreted differently by 3 different testers.

Scenario B: "Prose" Test

- **Steps:** "You go to the site, then you look where it says username and you write admin there, then you put in the password that was given to you and press the big orange login button."
- **QA Analysis:** Too much unnecessary text. Instructions should be telegraphic and technical: "1. Enter Username. 2. Enter Password. 3. Click Login."

Did you know that...?

💡 In the industry there is the concept of „**Step Overkill**“? Some testers write steps like "1. Open browser. 2. Type address. 3. Press Enter. 4. Wait for page to load."

In reality, these steps can be included in **PREREQUISITES**. The actual test should start where the targeted functionality begins. Saving reading time increases team productivity by up to 20%.

Applied Story: "The IKEA Assembly Manual"

Imagine a Test Case is a page from an IKEA assembly manual.

- **Preconditions:** You need to have all the parts out of the box and a screwdriver (Resources/Tools).
- **Steps:** There are clear drawings (Instructions) that tell you exactly which screw to put in which hole.
- **Test Data:** The 5mm screw (not the 10mm one).
- **Expected Result:** Two wooden boards are now joined at a 90 degree angle.

If the IKEA manual was poorly written (e.g. "Put the planks together"), your cabinet (the software) would come out crooked or fall apart. A professional tester writes perfect assembly manuals so that the software doesn't fall apart on first use.

Thinking Exercise: "Let's write correctly!"

We have the following requirement: "The system must allow password reset by sending a code

to email."

Task: Define **Preconditions** and **Expected Result** for this test case:

- **ID:** OHRM_AUTH_005
- **Title:** Verify receipt of password reset email.
- **Preconditions:** _____
- **Expected Result:** _____

Chapter 5: Types of Tests and the Testing Pyramid

 Software testing is not a one-size-fits-all activity. To be effective, we need to test the application at different “altitudes.” In this chapter, we explore the four main levels of testing, the strategic concept of the Testing Pyramid, and solve one of the biggest dilemmas for beginners: the difference between **Retesting** and **Regression**.

1. Testing Levels

According to ISTQB standards, testing is carried out in four logical stages as the software is built:

A. Unit Testing

- **What we test:** The smallest isolated components (functions, methods, code classes).

- **Who does it:** Developers.
- **Objective:** Checking the internal logic of the code. This is the cheapest stage of fixing bugs.

B. Integration Testing

- **What we test:** The interaction between two or more modules that work well individually, but may have problems when they "talk" to each other.
- **Who does it:** Developers or Technical Testers.
- **Objective:** Identifying defects in interfaces and data flows between components.

C. System Testing

- **What we test:** The entire system as a unitary whole, based on business requirements (SRS).
- **Who does it:** QA Testers (Manual or Automated).
- **Objective:** Verifying the application's compliance with the end user's needs. This is where End-to-End (E2E) testing comes in.

D. Acceptance Testing (UAT)

- **What we test:** Final validation of the system.
- **Who does it:** Customers, End Users or Product Owner.
- **Objective:** The decision to "accept" the product for production release. (The question is answered: "Is this product useful for my business?").

2. The Testing Pyramid

The Testing Pyramid concept (introduced by Mike Cohn) teaches us how to distribute testing effort to achieve the best Return on Investment (ROI).

- **Base (Unit Tests):** We need to have thousands of these tests. They're fast, automated, and give us instant feedback.
- **Middle (API/Integration Tests):** They check business logic without going through the graphical interface. They are slower than unitary ones, but faster than manual ones.
- **Top (UI/Manual Testing):** They are the most expensive and slowest tests. This is where QualiAdept manual testing comes in. Although they are few in number, they are vital to verifying the user experience (UX).

The strategic error: Many teams reverse the pyramid (relying only on manual testing at the end), which leads to huge costs and delayed releases.

3. Retesting vs. Regression Testing (The Big Distinction)

This is the "touchstone" of any QA interview.

Characteristic	Retesting (Confirmation Testing)	Regression Testing
Definition	We are checking whether a previously reported bug has actually been fixed.	We check if fixing a bug didn't break other areas that were working.
Scope	Confirmation of repair.	Protecting system integrity.
When is it done?	Right after the Developer says "Fixed".	After a successful Retest or at the end of the cycle.
Automation	Hard to automate (occasional bugs).	Ideal for automation (repetitive).

Practical Examples for Discussion (OrangeHRM)

Scenario A: "Save Button"

- **Context:** You reported that the "Save" button in the Personal Details module is not working. The developer is fixing it.
- **Action 1 (Retest):** Click on "Save" to see if it saves now. (It did!).
- **Action 2 (Regression):** Check if you can still edit your data or if you can still log out.
- **Discussion question:** Why might "Save" break "Logout"? (Answer: Script conflicts or corrupted sessions).

Scenario B: Testing levels in OrangeHRM

- **Unit:** A dev checks if the function that calculates the remaining vacation days returns $21 - 5 = 16$.
- **Integration:** We check if the "Leave" module correctly sends the notification to the "Dashboard" module.
- **System:** The QA tester does a complete flow: Hiring -> Leave Request -> Approval -> Payroll

Check.

Did you know that...?

💡 There is a type of test called „**Sanity Testing**“ ? It is a short form of regression that is done when we don't have time to test everything. If a full regression takes 2 days, a "Sanity" takes 2 hours and only checks the vital flows (e.g. "If we can't log in, we don't test the rest").

Applied Story: "Building a Smartphone"

Imagine you are the head of quality at a mobile phone factory.

- **Unit Testing:** Engineers check each part individually: Does the screen light up? Does the speaker make sound? Does the battery charge on the test bench?
- **Integration Testing:** We connect the screen to the motherboard. Do they both still work together? Or is the motherboard "burning" the screen?
- **System Testing:** We assemble the whole phone. We put the case on, turn on the Android and check: Can I make a call? Does the Wi-Fi work? Does the camera take clear pictures?
- **Acceptance Testing (UAT):** We send the phone to a group of regular users. They say, "It's nice, but it's too heavy," or "The volume button is too high." Based on their feedback, we decide whether to go into mass production.

Morals of the QualiAdept: If you skipped Unit Testing (didn't check the battery) and discovered that the battery exploded only during System Testing (when the phone is ready), you lost thousands of euros on cases and screens destroyed by a defective part. **Level testing means economy and safety!**

Thinking Exercise: "The Regression Detective"

An update has been made to the company logo in the header of the OrangeHRM page (a purely aesthetic change).

- What would you do? **Retesting**?
- What would you check at **Regression**? (Think: does the header appear on all pages? Could it cover the menu buttons on low resolutions?)

Bonus Chapter: Exploratory Testing vs. Scripted Testing (Intuition vs. Structure)

 In previous chapters, we learned how to write detailed test cases and how to plan everything “down to the last detail.” However, in reality, the most spectacular bugs are often found outside of these documents. In this bonus chapter, we explore the duality between **Scripted Testing** (formal process) and **Exploratory Testing** (the art of discovery).

1. Scripted Testing

This is what I've been practicing so far: writing Test Cases based on requirements, with precise steps and predefined expected results.

- **Advantages:**
 - Provides clear evidence of requirements coverage (Traceability).
 - It is easy to delegate and report (we know exactly how many tests passed/failed).
 - It is the basis for future automation.
- **Disadvantages:**
 - **The "tunnel" effect:** The tester only follows the written steps and may ignore an obvious bug that is an inch away from his click.
 - It takes a lot of time for documentation.

2. Exploratory Testing

Definition: A testing style where learning, test design, and execution occur simultaneously. The tester does not have a set of written steps, but a **Charter** – an overall objective.

- **Mechanism:** The tester explores the application based on experience, intuition, and what they discover "along the way."
- **Advantages:**
 - Find complex "edge case" bugs that no requirement anticipated.
 - It is extremely fast (no prior documentation required).
 - Stimulates creativity and a "detective" mindset.

Comparison: When do we use what?

Situation	Scripted Testing	Exploratory Testing
Objective	Compliance check (Does it meet the requirement?)	Discovering the Unknown (Where Can It Break?)
Documentation	Detailed Test Cases	Short notes, screenshots, logs
Moment	Regression, formal phases of System Testing	After scripted tests have passed, or in critical phases
Required skill	Attention to detail, rigor	Intuition, rich experience, lateral thinking

Practical Examples for Discussion (OrangeHRM)

Scenario A: Scripted (Personal Details Module)

- **Test Case:** "Enter the name 'Popescu', press Save, check if it was saved."
- **Result:** The test passes. Everything seems OK.

Scenario B: Exploratory (Personal Details Module)

- **Approach:** The tester thinks: "What happens if I try to save the name while quickly clicking

the Edit button of another field? Or if I put an emoji in the name?".

- **Result:** It discovers that the database does not support special characters and the application displays an ugly error code (Internal Server Error).
- **QualiAdept Conclusion:** The scripted test confirmed that it works. The exploratory test showed how it breaks.

Did you know that...?

💡 There is a method called **Session-Based Test Management (SBTM)**? This is the professional way to do exploratory testing: you set a timer (e.g. 90 minutes), choose a goal (e.g. "I will only explore the password reset flow"), and at the end you report what you learned and what bugs you found. It's not "play," it's disciplined exploration.

Applied Story: "The Tourist vs. the Explorer"

Imagine you are visiting a new city (Software Application).

- **The Tourist (Scripted Testing):** You have a tourist guide (Test Plan) and a list of objectives (Test Cases). You go to the Eiffel Tower, then to the Louvre, then to the Arc de Triomphe. See exactly what the book says. At the end, you can say: "I visited Paris according to plan."
- **The Explorer (Exploratory Testing):** You've seen the main sights, but then you decide to take a side street because you saw an interesting cat. You find a hidden bakery that makes the best baguette in the world, which no guidebook mentions.

Moral: As a QA, you have to be both. The Tourist makes sure we see the "stars" of the application (the requirements), and the Explorer finds the small (or big) problems hidden in places no one thought to look.

Thinking Exercise: "Beyond the Steps"

We have a scripted test case for **OrangeHRM**: "Inserting a profile picture."

The steps say: 1. Click 'Browse', 2. Select 'photo.jpg', 3. Click 'Upload'.

If you had 10 minutes **Exploratory Testing** on this functionality, what "side streets" would you explore?

- *Examples of thinking:* "What if I upload a 50GB file?", "What if I upload an .exe file renamed to .jpg?", "What if I press Upload 10 times in a row?".

Chapter 6: Recap, Conclusions and the OrangeHRM Project

 Congratulations! You've covered the fundamentals of planning and design in testing. If Session 1 taught you "why" we test, Session 2 gave you the answer to the question "how" we organize ourselves to be effective. A tester without a plan is like a traveler without a compass; they may reach their destination, but they will certainly waste a lot of time and energy along the way.

1. Comprehensive Review: What did we learn in Session 2?

- **Testing Plan:** Strategic project map. We learned to define **Scope** (what we test and what we don't), to establish **Entry/Exit Criteria** and assume clear roles in the team.
- **Requirements Analysis:** We have become documentation detectives. We know how to "hunt" for ambiguities and turn a vague requirement into a **Testing Condition** clear.
- **Test Case Design:** We learned the anatomy of a test case (ID, Preconditions, Steps, Expected Result). We know that a good test should be **atomic** and **reproducible**.
- **Levels and the Testing Pyramid:** I understood that the basis of quality lies in unit tests (done by developers), and we, the QA, check the system as a whole.
- **Retesting vs. Regression:** We've clarified the big dilemma. Retesting confirms the repair, Regression confirms that "nothing else broke."
- **Scripted vs. Exploratory Testing:** We learned to combine the rigor of written steps with the intuition of free discovery.

2. QualiAdept Conclusions: "The Laws of the QA Strategist"

- **The scope is your protection.** If it's not written in the Plan that you're testing on the iPhone

8, no one can blame you for not finding a bug there.

- **Ambiguous requirements give rise to definite bugs.** Clarify everything before the programmer writes the first line of code.
- **A Test Case without an Expected Result is just a suggestion.** The Expected Result is the only one that defines success or failure.
- **Regression is not optional.** The software is an interconnected organism; a change to "Login" can bring down "Payments" without any warning.

3. Homework: "OrangeHRM Project - Design Phase"

This theme is the beginning of your portfolio project. We will use the app **OrangeHRM** (Demo or locally installed version).

Task 1: Test Plan Fragment (PIM Module - Personal Information Management)

Define the following elements for testing the PIM module:

- **In-Scope:** Mention 3 functionalities that need to be tested.
- **Out-of-Scope:** Mention 2 elements that you decide NOT to test (e.g. a specific resolution, an unfinished sub-module).
- **Exit Criteria:** Write 2 conditions that must be met to be able to say: "We have finished testing the PIM module."

Task 2: Requirements Analysis (Case "Add Employee")

Request received from the client: *"The system must allow a new employee to be added by filling in their name and a unique ID."*

- Identify **2 ambiguities** in this requirement.
- Formulate **2 clarifying questions** that you would address to the Product Owner.

Task 3: Test Case Design (Execution)

Write **3 Test Cases** complete for functionality **Login** in OrangeHRM, using the structure: ID, Title, Preconditions, Steps, Test Data, Expected Result.

- **Test 1:** Happy Path (Successful Login).
- **Test 2:** Negative Path (Wrong password).
- **Test 3:** Edge Case/Validation (Empty Fields).

Task 4: Regression Strategy

Imagine that only the color of the "Login" button has changed.

- What do you do for **Retesting**?
- Choose **just one other way** from the application you would check at **Regression** and justify why you think it might be affected.

Final tip from the QualiAdept team

Don't try to write the "longest" Test Plan. Write the most useful one. In modern automation and testing, simplicity and clarity always beat quantity.

See you at **Session 3**, where we will learn how to report bugs we find using these plans!